Docker y Docker Compose con aplicación Java y MySQL

Trabajo en Clase 03 — Ingeniería de Software III

Diego López Camacho — Universidad Industrial de Santander

30 de septiembre de 2026

1. Objetivo y entorno

El objetivo del trabajo es empaquetar la aplicación `demo-academico` —el proyecto Spring Boot que venimos construyendo desde el Lab 02— dentro de una imagen Docker, y después coordinarla junto a una base de datos MySQL usando Docker Compose.

El entorno sobre el que se ejecutaron todas las pruebas es el siguiente:

Componente	Versión
Docker (cliente)	29.4.0
Docker (servidor)	29.2.1
Sistema del daemon	Ubuntu 24.04.4 LTS
Arquitectura	aarch64 (Apple Silicon)
Java	OpenJDK 21.0.9
Maven	3.9.12
Aplicación	demo-academico 0.0.1-SNAPSHOT (Spring Boot 3.3.4)

Toda la salida de consola que se cita en este informe está recogida íntegra en el archivo `evidencia.txt` que acompaña al proyecto.

Una desviación respecto al enunciado, y por qué

El enunciado publica la aplicación en el puerto `8080` del host. En el equipo donde se desarrolló el laboratorio ese puerto ya estaba ocupado por otro contenedor ajeno a esta materia, de modo que **la aplicación se publicó en el puerto 8081 del host**, manteniendo el 8080 dentro del contenedor:

```
-p 8081:8080     y     ports: - "8081:8080"
```

La desviación resultó útil: obligó a aplicar exactamente el concepto que explica la sección 10 del enunciado. El puerto interno del contenedor no cambió, solo cambió por cuál puerto del host se accede a él.

2. Entregable 1 — Contenedor de prueba

```
$ docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (arm64v8)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.
```

Lo relevante de esta salida no es el saludo sino la secuencia que describe: el cliente habla con el daemon, el daemon **descarga una imagen**, crea **un contenedor** a partir de ella y lo ejecuta. También se confirma que la imagen descargada es la variante `arm64v8`, correspondiente a la arquitectura del equipo: Docker Hub resolvió automáticamente la variante adecuada desde un manifiesto multiarquitectura.

El contenedor imprime su mensaje y termina. Esto hace visible algo que suele confundirse al empezar: un contenedor no es un servicio que queda encendido, es un proceso. Si el proceso principal termina, el contenedor termina con él. Los contenedores que “quedan corriendo” —como nginx o nuestra aplicación— lo hacen porque su proceso principal es un servidor que no retorna.

3. Entregable 2 — Servidor nginx y mapeo de puertos

```
$ docker run -d -p 8088:80 --name mi-nginx nginx
a7bbd5c250714372b055369725fd01763743f45deb0dd8ab2fb842528756a11d

$ curl -s -o /dev/null -w 'HTTP %{http_code}' http://localhost:8088
HTTP 200

$ curl -s http://localhost:8088 | head -5
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
```

Levantando dos contenedores del mismo tipo en puertos distintos del host:

```
$ docker run -d -p 8088:80 --name nginx1 nginx
$ docker run -d -p 8089:80 --name nginx2 nginx

  8088 -> HTTP 200
  8089 -> HTTP 200

$ docker ps --filter name=nginx
NAMES     IMAGE     PORTS
nginx2    nginx     0.0.0.0:8089->80/tcp, [::]:8089->80/tcp
nginx1    nginx     0.0.0.0:8088->80/tcp, [::]:8088->80/tcp
```

Ambos contenedores escuchan internamente en el **puerto 80**, el mismo, y no hay ningún conflicto: cada uno está aislado en su propio espacio de red. El conflicto solo puede darse del lado del host, porque ahí sí hay un único espacio de puertos compartido. Para comprobarlo se intentó publicar un tercer nginx en un puerto del host ya tomado:

```
$ docker run -d -p 8088:80 --name nginx3 nginx
docker: Error response from daemon: failed to set up container networking:
driver failed programming external connectivity on endpoint nginx3:
Bind for 0.0.0.0:8088 failed: port is already allocated
```

El error es explícito: `port is already allocated`. Es exactamente la situación que obligó a usar el 8081 para la aplicación.

4. Entregable 3 — Construcción de la imagen

4.1 Compilación previa

```
$ mvn clean package
[INFO] Building jar: .../target/demo-academico-0.0.1-SNAPSHOT.jar
[INFO] Replacing main artifact ... with repackaged archive,
       adding nested dependencies in BOOT-INF/.
[INFO] BUILD SUCCESS

$ ls -lh target/*.jar
53M target/demo-academico-0.0.1-SNAPSHOT.jar
```

El artefacto es un *fat jar* de 53 MB: Spring Boot empaqueta la aplicación junto con todas sus dependencias y un Tomcat embebido, de modo que el jar se ejecuta por sí solo. Ese es el archivo que la imagen va a copiar.

4.2 El Dockerfile

```
FROM eclipse-temurin:21-jdk
RUN mkdir -p /opt/demo-academico
ENV TZ="America/Bogota"
ENV WORKSPACE /opt/demo-academico
WORKDIR ${WORKSPACE}
COPY target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java","-Xmx1024m","-jar","app.jar"]
```

Conviene detenerse en dos instrucciones que se malinterpretan con frecuencia. `EXPOSE 8080` **no publica nada**: es documentación dentro de la imagen que declara en qué puerto escucha el proceso. Para que ese puerto sea alcanzable desde el host hace falta `-p` al ejecutar, o `ports:` en Compose. Y `ENTRYPOINT` define el proceso principal del contenedor: cuando ese proceso muera, el contenedor muere.

4.3 Construcción

```
$ docker build -t demoacademico .
Step 5/8 : WORKDIR ${WORKSPACE}
Step 6/8 : COPY target/*.jar app.jar
Step 7/8 : EXPOSE 8080
Step 8/8 : ENTRYPOINT ["java","-Xmx1024m","-jar","app.jar"]
Successfully built 225e93eaef24
Successfully tagged demoacademico:latest

real 6.04 s
$ docker images
REPOSITORY        TAG                 IMAGE ID       SIZE
demoacademico     latest              225e93eaef24   862MB
nginx             latest              f9ea18bfa4fa   261MB
eclipse-temurin   21-jdk              f9bf7e9fe2f9   762MB
mysql             8.0                 7dcddc01f13b   1.09GB
```

La construcción tardó 6 segundos porque lo único que hace es copiar un jar ya compilado sobre una imagen base ya descargada. El dato que más llama la atención es el tamaño: **862 MB**, de los cuales 762 MB son la imagen base `eclipse-temurin:21-jdk` y unos 53 MB el jar. Es decir, el 88 % del peso de nuestra imagen no es nuestra aplicación, sino el JDK completo que viene debajo. Esta observación es el punto de partida del laboratorio siguiente sobre optimización de imágenes.

5. Entregable 4 — Contenedor de la aplicación

```
$ docker run -d -p 8081:8080 --name demoacademico demoacademico
b9a097ffb93db088afd40ff72489807e969976ab900f684197d06ee3e150c7f8

$ docker ps
NAMES           IMAGE           STATUS          PORTS
demoacademico   demoacademico   Up 25 seconds   0.0.0.0:8081->8080/tcp
$ docker logs demoacademico
2026-10-14T14:30:31.714-05:00  INFO 1 --- [main] TomcatWebServer :
    Tomcat started on port 8080 (http) with context path '/'
2026-10-14T14:30:31.720-05:00  INFO 1 --- [main] DemoAcademicoApplication :
    Started DemoAcademicoApplication in 2.02 seconds
```

Dos detalles de esta salida merecen comentario. El primero es que las marcas de tiempo llevan desplazamiento `-05:00`, lo que confirma que la instrucción `ENV TZ="America/Bogota"` del Dockerfile surtió efecto dentro del contenedor. El segundo es el 1 en `INFO 1 ---`: ese es el PID del proceso Java, que dentro del contenedor es **1**, el proceso raíz. Si ese proceso termina, el contenedor termina.

5.1 Prueba del servicio

```
$ curl "http://localhost:8081/api/estudiantes?page=0&size=2"
{"success":true,"message":"OK","data":{"content":[
  {"id":1,"nombre":"Sta. María Soledad Arreola Caballero", ...},
  {"id":2,"nombre":"José Luis Valadez Delarosa", ...}],
```

```
  "totalElements":500,"totalPages":250,"size":2,"number":0, ...}}
HTTP 200

$ curl http://localhost:8081/swagger-ui/index.html
swagger-ui -> HTTP 200

$ POST /api/estudiantes  {"nombre":"Ana Perez","email":"ana.perez@clase03.com"}
{"success":true,"message":"Estudiante creado","data":{"id":501, ...}}
HTTP 201

$ POST /api/estudiantes  (mismo email, error controlado)
{"success":false,"message":"Ya existe un estudiante con ese email","data":null}
HTTP 400
```

La aplicación responde igual que fuera del contenedor: listado paginado con los 500 estudiantes del seeder, Swagger accesible, creación correcta y la regla de email único devolviendo un error controlado en lugar de un stacktrace.

6. Entregable 5 — Docker Compose

```
services:
  app:
    image: demoacademico
    container_name: demoacademico-app
    ports:
      - "8081:8080"      # el enunciado usa 8080:8080; aquí el 8080 está ocupado
    depends_on:
      - db

  db:
    image: mysql:8.0
    container_name: demoacademico-db
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: demoacademico
      MYSQL_USER: demo
      MYSQL_PASSWORD: demo123
    ports:
      - "3307:3306"
    volumes:
      - mysql_data:/var/lib/mysql

volumes:
  mysql_data:
```

```
$ docker compose up -d
 Volume clase03_mysql_data   Created
 Container demoacademico-db  Created   →  Started
 Container demoacademico-app Created   →  Started

$ docker compose ps
NAME                IMAGE           SERVICE   STATUS         PORTS
demoacademico-app   demoacademico   app       Up 30 seconds  0.0.0.0:8081->8080/tcp
demoacademico-db    mysql:8.0       db        Up 30 seconds  0.0.0.0:3307->3306/tcp, 33060/tcp
$ docker logs demoacademico-db
[Entrypoint]: Creating database demoacademico
[Entrypoint]: Creating user demo
[Server] /usr/sbin/mysqld: ready for connections. Version: '8.0.46'

$ docker exec demoacademico-db mysql -udemo -pdemo123 -e 'SHOW DATABASES;'
Database
demoacademico
information_schema
performance_schema
```

Las variables de entorno del servicio `db` se aplicaron: MySQL creó automáticamente la base `demoacademico` y el usuario `demo` en el primer arranque. Un solo comando levantó los dos servicios, creó el volumen y creó una red para ellos.

6.1 Observación: la aplicación no está usando este MySQL

Al revisar los registros apareció algo que conviene documentar con honestidad, porque no es un fallo del laboratorio sino una consecuencia de lo que el archivo declara:

```
$ docker exec demoacademico-app env | grep -i spring
  variables SPRING_* definidas: 0

$ docker logs demoacademico-app | grep -i 'h2\|jdbc'
  H2 console available at '/h2-console'.
  jdbc:h2:mem:demoacademico

$ docker exec demoacademico-db mysql -udemo -pdemo123 demoacademico -e 'SHOW TABLES;'
Tables_in_demoacademico
prueba_persistencia
```

La aplicación sigue conectada a **H2 en memoria**, porque el `docker-compose.yml` no le pasa ninguna variable de datasource. En MySQL no existe la tabla `estudiante`: la única tabla es la que se creó a mano para probar el volumen. Los dos servicios están levantados y coordinados, pero todavía no están integrados.

`depends_on` tampoco cierra esa brecha, porque solo controla el **orden de arranque**, no la conexión ni la disponibilidad real: inicia `db` antes que `app`, pero no espera a que MySQL esté listo para aceptar conexiones. Conectar de verdad la aplicación exigiría añadir al servicio `app` variables como `SPRING_DATASOURCE_URL: jdbc:mysql://db:3306/demoacademico`, usando el nombre del servicio `db` como host —tema del laboratorio siguiente sobre redes— y cambiar el driver de H2 a MySQL en el `pom.xml`.

7. Entregable 6 — Volúmenes y persistencia

```
$ docker volume ls
DRIVER    VOLUME NAME
local     clase03_mysql_data
```

```
$ docker volume inspect clase03_mysql_data
  Name: clase03_mysql_data
  Driver: local
  Mountpoint: /var/lib/docker/volumes/clase03_mysql_data/_data
```

Para comprobar que el volumen realmente desacopla los datos del ciclo de vida del contenedor, se escribió un registro, se destruyeron los contenedores y se volvieron a crear:

```
$ mysql> CREATE TABLE prueba_persistencia (id INT PRIMARY KEY, nota VARCHAR(80));
$ mysql> INSERT INTO prueba_persistencia VALUES (1, 'escrito antes del compose down');
id      nota
1       escrito antes del compose down

$ docker compose down
 Container demoacademico-app  Removed
 Container demoacademico-db   Removed
 Network   clase03_default    Removed

$ docker ps -a --filter name=demoacademico
  (ninguno: los contenedores fueron eliminados)

$ docker volume ls | grep clase03
local     clase03_mysql_data        ←  el volumen sobrevive

$ docker compose up -d
$ mysql> SELECT * FROM prueba_persistencia;
id      nota
1       escrito antes del compose down        ←  el dato sigue ahí
```

Los contenedores fueron eliminados por completo y recreados desde la imagen, y el dato seguía presente. Nótese además que `docker compose down` eliminó los contenedores y la red, pero **no el volumen**: ese es el comportamiento por defecto, y es deliberado, porque borrar datos no debería ser un efecto secundario de apagar un servicio. Para eliminarlo hay que pedirlo explícitamente, con `docker compose down -v` o `docker volume rm`.

8. Entregable 7 — Reflexión técnica

¿Cuál es la diferencia entre imagen y contenedor?

La imagen es una plantilla inmutable de solo lectura: el sistema base, las dependencias y la aplicación, en capas. El contenedor es una instancia en ejecución de esa plantilla, con una capa de escritura propia encima. La relación es la misma que entre una clase y un objeto: de la imagen `demoacademico` (862 MB, un único identificador `225e93eaef24`) se pueden crear tantos contenedores como se quiera, y cada uno tendrá su propio sistema de archivos modificable, su propio espacio de red y su propio ciclo de vida. Lo vimos de forma directa con nginx: una sola imagen, dos contenedores (`nginx1` y `nginx2`) ejecutándose a la vez sin interferir.

¿Qué problema de ingeniería de software ayuda a resolver Docker?

El problema del "en mi computador funciona", que no es un problema de descuido sino de **deriva de configuración** entre entornos: versiones distintas de Java, librerías del sistema, variables de entorno, zonas horarias. Docker lo resuelve haciendo que el entorno de ejecución deje de ser algo que se configura en cada máquina y pase a ser algo que se **declara en un archivo versionado** junto al código. El Dockerfile de este laboratorio fija Java 21 y la zona horaria de Bogotá; cualquiera que construya esa imagen obtiene exactamente el mismo entorno, sin instalar nada. En términos de ingeniería de software, convierte la configuración de infraestructura en código revisable, reproducible y sujeto a control de versiones.

¿Qué ventaja ofrece Docker Compose frente a ejecutar contenedores manualmente?

Pasa de una secuencia de comandos imperativos a una **descripción declarativa**. Levantar los dos servicios a mano exige recordar y ejecutar en el orden correcto los `docker run` con todos sus `-p`, `-e`, `-v` y `--network`; con Compose, toda esa configuración queda en un archivo que se versiona junto al código y se levanta con `docker compose up -d`. La diferencia práctica se nota en tres cosas que observamos: Compose creó automáticamente una red para que los servicios se encuentren por nombre, gestionó el orden de arranque con `depends_on`, y nombró el volumen con el prefijo del proyecto (`clase03_mysql_data`) para aislarlo de otros proyectos. Además, `docker compose down` deshace todo el conjunto de una vez, sin tener que recordar qué se creó.

¿Por qué es importante usar volúmenes cuando se trabaja con bases de datos?

Porque sin volumen los datos viven en la capa de escritura del contenedor, que se destruye junto con él. Un contenedor es, por diseño, desechable: se recrea para actualizar la imagen, para cambiar configuración o simplemente porque falló. Si los datos estuvieran dentro, cada una de esas operaciones rutinarias sería una pérdida total de información. El volumen rompe esa dependencia, y lo comprobamos: eliminamos por completo el contenedor de MySQL y el dato siguió existiendo. La regla general es que el contenedor debe poder destruirse en cualquier momento sin consecuencias, y eso solo es cierto si el estado vive fuera de él.

¿Qué significa el mapeo de puertos 8080:8080 y 3307:3306?

La sintaxis es `puertoHost:puertoContenedor`, y la dirección importa. En `3307:3306`, MySQL escucha internamente en su puerto estándar 3306 —dentro del contenedor nada cambió— y Docker redirige hacia él el puerto 3307 del host. Se usa 3307 y no 3306 precisamente para no chocar con un MySQL que pudiera estar instalado en la máquina. En `8080:8080` ambos números coinciden, pero siguen siendo dos espacios distintos: el de la izquierda es del host y el de la derecha del contenedor. Que coincidan es una comodidad, no una obligación, y en este laboratorio tuvimos que romper esa coincidencia y usar `8081:8080` porque el 8080 del host ya estaba ocupado. Los contenedores no comparten espacio de puertos entre sí —`nginx1` y `nginx2` escucharon ambos en el 80 sin problema—, pero sí compiten por los puertos del host, que son un recurso único.

9. Conclusión

El laboratorio cumplió su objetivo: la aplicación quedó empaquetada en una imagen reproducible y se levantó junto a MySQL con un solo comando. Más allá de los pasos, quedan tres aprendizajes que vale la pena señalar.

El primero es que **el aislamiento es real pero el host es compartido**. Dos contenedores pueden usar el mismo puerto interno sin problema, y el conflicto solo aparece al publicar hacia afuera. Tener que desplazar la aplicación al 8081 por un puerto ocupado hizo tangible esa distinción mejor que cualquier explicación.

El segundo es que **el tamaño de la imagen no es un detalle**. 862 MB para una aplicación de 53 MB significa que el 88 % de lo que se transfiere y almacena en cada despliegue es entorno de construcción que probablemente no se necesita en ejecución.

El tercero, y el más útil, es que **levantar servicios juntos no es integrarlos**. Compose arrancó la aplicación y MySQL de forma coordinada, pero la aplicación siguió usando H2 porque nadie le dijo lo contrario, y `depends_on` solo ordenó el arranque sin garantizar disponibilidad. Es una distinción fácil de pasar por alto cuando `docker compose ps` muestra todo en `Up` y se asume que el sistema está completo.